

EJERCICIO3_R

Carga de datos

Comenzamos cargando las paqueterías y librerías necesarias, así como la base de datos que se utilizará en esta tercera parte del proyecto.

```
# Instalamos las librerías necesarias para R.  
  
#install.packages(c("dplyr", "ggplot2", "janitor", "inspectdf", "lubridate", "nanian", "readr", "r  
  
library(dplyr)
```

Warning: package 'dplyr' was built under R version 4.3.3

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
library(ggplot2)  
library(janitor)
```

Warning: package 'janitor' was built under R version 4.3.3

Attaching package: 'janitor'

The following objects are masked from 'package:stats':

chisq.test, fisher.test

```
library(inspectdf)
```

Warning: package 'inspectdf' was built under R version 4.3.3

```
library(lubridate)
```

Warning: package 'lubridate' was built under R version 4.3.3

Attaching package: 'lubridate'

The following objects are masked from 'package:base':

date, intersect, setdiff, union

```
library(naniar)
```

Warning: package 'naniar' was built under R version 4.3.3

```
library(readr)
```

Warning: package 'readr' was built under R version 4.3.3

```
library(reticulate)
```

Warning: package 'reticulate' was built under R version 4.3.3

```
library(stringr)
library(tidyr)
```

Warning: package 'tidyr' was built under R version 4.3.3

```
library(visdat)
```

Warning: package 'visdat' was built under R version 4.3.3

```
# Instalamos las librerías necesarias para Python.
```

```
py_install(c("pandas", "seaborn", "numpy", "matplotlib"))
```

Using virtual environment "C:/Users/23axe/Documents/.virtualenvs/r-reticulate" ...

```
+ "C:/Users/23axe/Documents/.virtualenvs/r-reticulate/Scripts/python.exe" -m pip install --
upgrade --no-user pandas seaborn numpy matplotlib
```

```
# Cargamos nuestra base de datos "coffe_ratings".
```

```
url <- "https://raw.githubusercontent.com/EduardoSelimMM/20260101_LecturaLimpiezaDatos/main/proye
```

```
datos <- read_csv(url)
```

Rows: 1339 Columns: 43

— Column specification —

Delimiter: ",",

```
chr (24): species, owner, country_of_origin, farm_name, lot_number, mill, ic...
dbl (19): total_cup_points, number_of_bags, aroma, flavor, aftertaste, acidi...
```

- ❖ Use ``spec()`` to retrieve the full column specification for this data.
- ❖ Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

Actividad 0:

Realice un análisis de valores faltantes del objeto `datos`

Para ello empezaremos con el código en R.

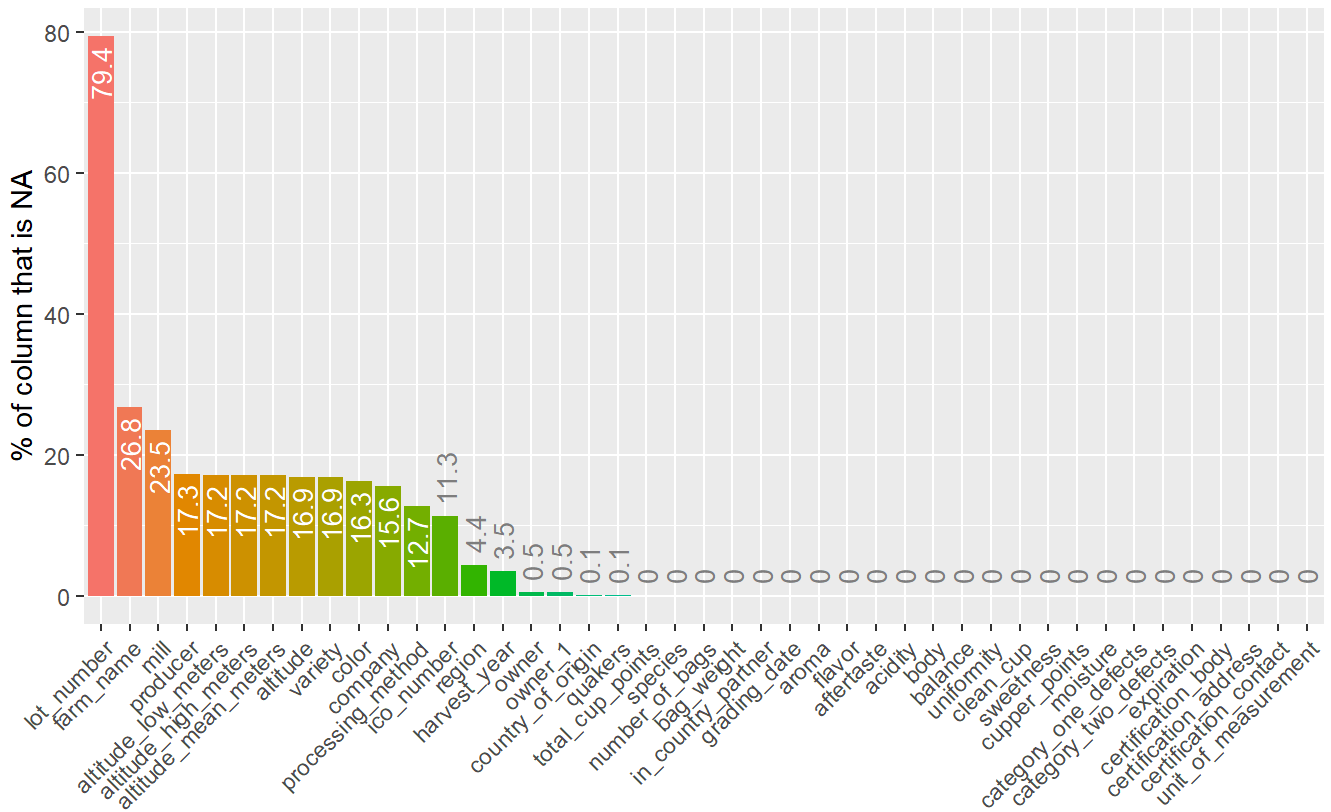
```
# Hacemos un análisis de los datos faltantes de nuestra base de datos.
miss_var_summary(datos)
```

```
# A tibble: 43 × 3
  variable      n_miss pct_miss
  <chr>         <int>   <num>
1 lot_number    1063    79.4
2 farm_name     359    26.8
3 mill          315    23.5
4 producer     231    17.3
5 altitude_low_meters 230    17.2
6 altitude_high_meters 230    17.2
7 altitude_mean_meters 230    17.2
8 altitude      226    16.9
9 variety       226    16.9
10 color        218    16.3
# i 33 more rows
```

```
# Hacemos una visualización de nuestros datos faltantes.
show_plot(inspect_na(datos))
```

Prevalence of NAs in df::datos

df::datos has 43 columns, of which 19 have missing values



Ahora se realiza el análisis de valores faltantes del objeto `datos` en Python.

```
import pandas as pd
import seaborn as sns
import re
import numpy as np

sns.set(style="whitegrid")

url = "https://raw.githubusercontent.com/EduardoSelimMM/20260101_LecturaLimpiezaDatos/main/proyec

datos = pd.read_csv(url)

faltantes = datos.isnull().sum().reset_index()
faltantes.columns = ["variable", "n_faltantes"]

# Porcentaje

faltantes["pct_faltantes"] = (faltantes["n_faltantes"] / len(datos)) * 100

# Filtrar solo los que tienen valores faltantes

faltantes = faltantes[faltantes["n_faltantes"] > 0]
```

```
# Ordenar
```

```
faltantes = faltantes.sort_values(by="n_faltantes", ascending=False)
```

```
faltantes
```

	variable	n_faltantes	pct_faltantes
5	lot_number	1063	79.387603
4	farm_name	359	26.811053
6	mill	318	23.749066
33	color	270	20.164302
11	producer	232	17.326363
41	altitude_high_meters	230	17.176998
40	altitude_low_meters	230	17.176998
42	altitude_mean_meters	230	17.176998
9	altitude	226	16.878267
18	variety	226	16.878267
8	company	209	15.608663
19	processing_method	170	12.696042
7	ico_number	159	11.874533
10	region	59	4.406273
15	harvest_year	47	3.510082
17	owner_1	7	0.522778
2	owner	7	0.522778
3	country_of_origin	1	0.074683
32	quakers	1	0.074683

```
import matplotlib.pyplot as plt
```

```
plt.figure()
```

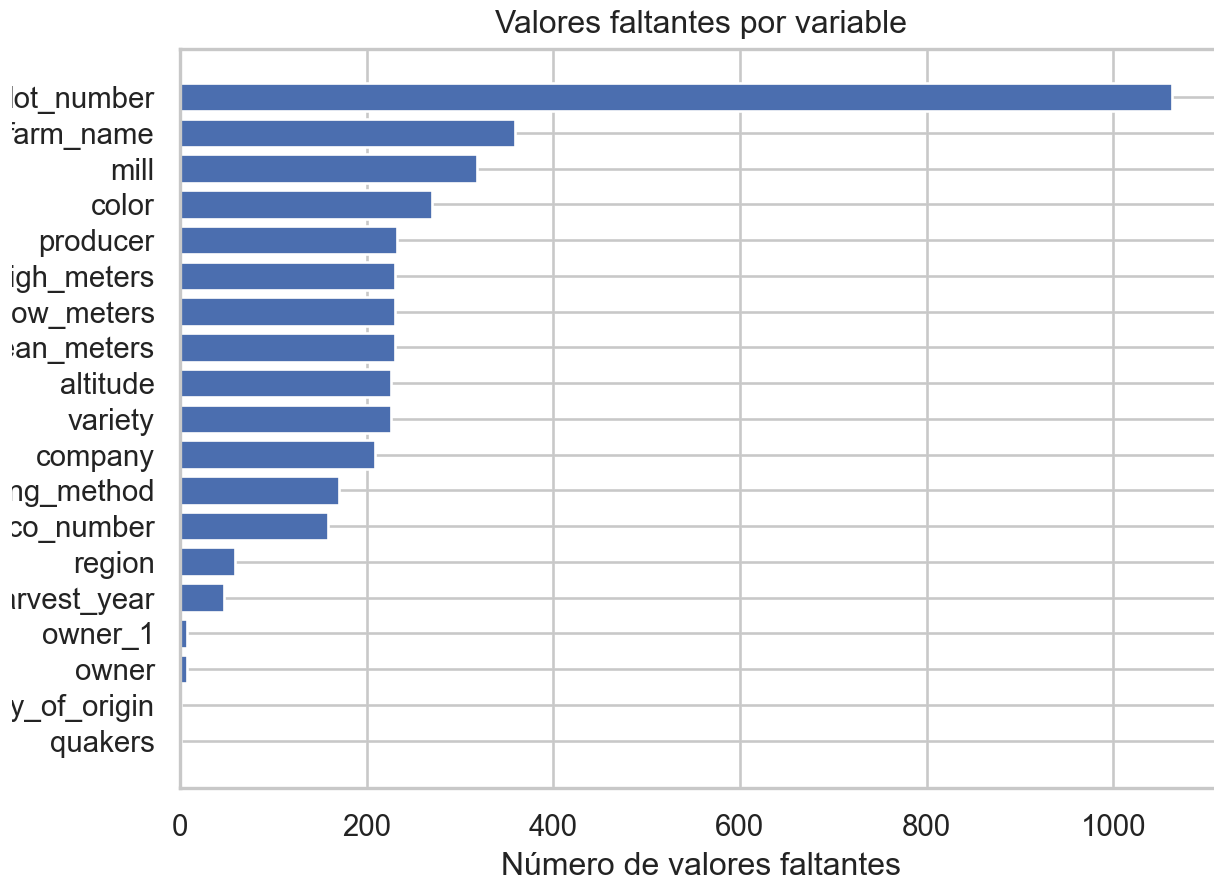
```
# Asegurar orden descendente y luego invertir para barh
```

```
faltantes_plot = faltantes.sort_values(by="n_faltantes", ascending=False)
```

```
plt.barh(  
    faltantes_plot["variable"][::-1],  
    faltantes_plot["n_faltantes"][::-1]  
)
```

```
plt.xlabel("Número de valores faltantes")
plt.ylabel("Variables")
plt.title("Valores faltantes por variable")

plt.show()
```



En ambos casos, podemos observar que, de las 43 variables presentes en la base de datos, 19 contienen valores faltantes (NA). Destaca la variable `lot_number`, que presenta un número considerablemente alto de valores ausentes (1063), lo que limita su utilidad para nuestro análisis. De igual forma, variables como `farm_name` y `mill` muestran niveles considerables de incompletitud.

Por otro lado, variables relacionadas con la altitud, presentan números similares de valores faltantes, lo que podría suponer la existencia de un patrón en la ausencia de datos.

Actividad 1:

Crear una columna llamada `color2` que se base en los valores de la columna `color`, que asigne el valor NA si `color == NA`, `"#00FF66"` si `color == 'Green'`, `"#CCEBC5"` si `color == 'Bluish-green'` y `"#BFFFFFFF"` si `color == 'Blue-green'`

Comenzamos con la implementación en R.

Antes de empezar le daremos un breve vistazo a la columna *color* para identificar el formato de sus registros.

```
# Datos únicos de la columna color.  
unique(datos$color)
```

```
[1] "Green"      NA           "Bluish-Green" "None"       "Blue-Green"
```

Podemos observar que la columna contiene los valores "Green", NA, "Bluish-Green", "None" y "Blue-Green"; a partir de esta información, es posible construir la columna *color2*.

```
# Empezamos haciendo un diccionario con cada una de las palabras clave y el color que le corresponde  
colores <- c(  
  "green" = "#00FF66",  
  "bluish-green" = "#CCEBC5",  
  "blue-green" = "#BFFFFFFF"  
)  
  
# Hacemos una columna llamada color2 con base en los valores de la columna color y el diccionario  
datos <- datos |>  
  mutate(  
    color_limpio = str_to_lower(color),  
    color2 = colores[color_limpio]  
  )  
  
# Mostramos los registros de la columna "color2"  
datos |> select(color2)
```

```
# A tibble: 1,339 × 1  
  color2  
  <chr>  
1 #00FF66  
2 #00FF66  
3 <NA>  
4 #00FF66  
5 #00FF66  
6 #CCEBC5  
7 #CCEBC5  
8 <NA>  
9 <NA>  
10 #00FF66  
# i 1,329 more rows
```

A continuación, se implementa el mismo procedimiento en Python.

Previo a ello, se realiza una revisión rápida de la columna *color* con el fin de identificar el formato de sus valores.

```
# Ver valores únicos de la columna color
```

```
print(datos["color"].unique())
```

```
<StringArray>
```

```
['Green', nan, 'Bluish-Green', 'Blue-Green']
```

```
Length: 4, dtype: str
```

```
# Crear diccionario de colores
```

```
colores = {  
    "green": "#00FF66",  
    "bluish-green": "#CCEBC5",  
    "blue-green": "#BFFFFFFF"  
}
```

Se identifica que la columna presenta los valores "Green", NA, "Bluish-Green", "None" y "Blue-Green"; con base en estos registros, se puede generar la nueva columna `color2`.

```
# Diccionario de colores
```

```
colores = {  
    "green": "#00FF66",  
    "bluish-green": "#CCEBC5",  
    "blue-green": "#BFFFFFFF"  
}
```

```
# Crear columna color2 basada en color
```

```
datos["color_limpio"] = datos["color"].str.lower()  
datos["color2"] = datos["color_limpio"].map(colores)
```

```
# Mostrar solo las primeras 50 filas de color2
```

```
print(datos[["color2"]].head(50))
```

```
      color2  
0  #00FF66  
1  #00FF66  
2      NaN  
3  #00FF66  
4  #00FF66  
5  #CCEBC5  
6  #CCEBC5  
7      NaN  
8      NaN  
9  #00FF66  
10     NaN
```

11	NaN
12	NaN
13	NaN
14	NaN
15	NaN
16	NaN
17	NaN
18	#00FF66
19	NaN
20	NaN
21	#00FF66
22	#00FF66
23	NaN
24	#CCEBC5
25	#00FF66
26	NaN
27	#00FF66
28	#00FF66
29	#00FF66
30	NaN
31	#CCEBC5
32	#00FF66
33	#BFFFFFF
34	#00FF66
35	#BFFFFFF
36	NaN
37	NaN
38	NaN
39	NaN
40	#00FF66
41	NaN
42	NaN
43	#00FF66
44	#CCEBC5
45	#BFFFFFF
46	NaN
47	#00FF66
48	#00FF66
49	#BFFFFFF

En este caso solo mostramos los primeros 50 datos para no saturar de informacion, sin embargo en estos 50 datos podemos observar el cambio con su respectivo color.

Es importante destacar que tanto en R como en Python, todos los valores de color se convirtieron a minúsculas con el fin de estandarizar el formato y evitar inconsistencias al momento de asignar un color.

Actividad 2:

Crear una columna llamada `bag_weight2` que se base en los valores de la columna `bag_weight`, que sólo contenga el valor numérico de ésta. Es decir, `bag_weight2` debe ser numérica. ¿Cuántas observaciones llevaron a ambigüedad para crear esta nueva columna?

Se comienza con la implementación en R.

Se empieza con un análisis exploratorio de la columna `bag_weight`, con el objetivo de identificar los valores que contiene.

```
# Datos únicos de la columna bag_weight.  
unique(datos$bag_weight)
```

```
[1] "60 kg"    "1"        "30 kg"    "69 kg"    "1 kg"     "2 kg, lbs"  
[7] "6"        "3 lbs"    "50 kg"    "2 lbs"    "100 lbs"  "15 kg"  
[13] "2 kg"     "2"        "70 kg"    "19200 kg" "5 lbs"    "1 kg, lbs"  
[19] "6 kg"     "0 lbs"    "46 kg"    "40 kg"    "20 kg"    "34 kg"  
[25] "1 lbs"    "660 kg"   "18975 kg" "12000 kg" "35 kg"    "66 kg"  
[31] "80 kg"    "132 lbs"  "5 kg"     "25 kg"    "59 kg"    "18000 kg"  
[37] "150 lbs"  "9000 kg"  "18 kg"    "10 kg"    "29 kg"    "1218 kg"  
[43] "4 lbs"    "0 kg"     "13800 kg" "1500 kg"  "24 kg"    "80 lbs"  
[49] "8 kg"     "3 kg"     "350 kg"   "67 kg"    "4 kg"     "55 lbs"  
[55] "100 kg"   "130 lbs"
```

Se observa que algunos registros no cuentan con unidad de medida, presentan múltiples unidades, toman el valor de cero o contienen valores atípicos.

Por lo tanto, esta información se considerará en la construcción de la nueva columna `bag_weight2`. En particular, se realizará la conversión de libras a kilogramos con el fin de unificar las unidades de medida en la base de datos. Asimismo, aquellos registros que presenten más de una unidad de medida o que no cuenten con ninguna serán asignados como NA, ya que constituyen casos ambiguos que no pueden interpretarse con certeza y podrían generar inconsistencias en el análisis posterior.

```
# Creamos las nuevas columnas.  
datos <- datos |>  
  mutate(  
    # Extraemos número  
    valor = as.numeric(str_extract(bag_weight, "\\d+")),  
  
    # Detectamos la unidad de medida y convertimos lb a kg si es necesario  
    bag_weight2 = case_when(  
      str_detect(bag_weight, "kg") & !str_detect(bag_weight, "lbs") ~ valor,  
      str_detect(bag_weight, "lbs") & !str_detect(bag_weight, "kg") ~ valor * 0.45359237,  
      TRUE ~ NA_real_  
    )  
  ) |>  
  select(-valor)  
  
# Observamos los resultados obtenidos.  
datos |>
```

```
select(bag_weight, bag_weight2) |>
head(10)
```

```
# A tibble: 10 × 2
  bag_weight bag_weight2
  <chr>      <dbl>
1 60 kg      60
2 60 kg      60
3 1          NA
4 60 kg      60
5 60 kg      60
6 30 kg      30
7 69 kg      69
8 60 kg      60
9 60 kg      60
10 60 kg     60
```

Podemos observar que la nueva columna extrae correctamente el valor numérico de la columna `bag_weight`.

Ahora nos preguntamos, ¿Cuántas observaciones llevaron a ambigüedad para crear esta nueva columna?

Para ello, consideraremos como valores atípicos aquellos registros que no cuentan con una unidad de medida, así como aquellos que presentan múltiples unidades.

```
# Filtramos aquellos datos ambiguos.
ambigüedad <- datos |>
  filter(
    !is.na(bag_weight) &
    (
      str_detect(bag_weight, "^[0-9]+$") | # sin unidad
      str_detect(bag_weight, ",")         # múltiples unidades
    )
  )

# Contamos el número de datos ambiguos.
nrow(ambigüedad)
```

```
[1] 29
```

A continuación, se replica el mismo ejercicio en Python.

Como primer paso, se realiza un análisis exploratorio de la columna `bag_weight` para identificar los tipos de valores que contiene.

```
print(datos["bag_weight"].unique())
```

```
<StringArray>
[ '60 kg',      '1',      '30 kg',      '69 kg',      '1 kg', '2 kg,1bs',
```

```

'6',      '3 lbs',      '50 kg',      '2 lbs',      '100 lbs',      '15 kg',
'2 kg',      '2',      '70 kg',      '19200 kg',      '5 lbs',      '1 kg,lbs',
'6 kg',      '0 lbs',      '46 kg',      '40 kg',      '20 kg',      '34 kg',
'1 lbs',      '660 kg',      '18975 kg',      '12000 kg',      '35 kg',      '66 kg',
'80 kg',      '132 lbs',      '5 kg',      '25 kg',      '59 kg',      '18000 kg',
'150 lbs',      '9000 kg',      '18 kg',      '10 kg',      '29 kg',      '1218 kg',
'4 lbs',      '0 kg',      '13800 kg',      '1500 kg',      '24 kg',      '80 lbs',
'8 kg',      '3 kg',      '350 kg',      '67 kg',      '4 kg',      '55 lbs',
'100 kg',      '130 lbs']

```

Length: 56, dtype: str

Se logra identificar que ciertos registros carecen de unidad de medida, incluyen más de una unidad, toman el valor cero o presentan valores atípicos.

En consecuencia, en la construcción de la columna `bag_weight2`, se convertirán las libras a kilogramos para unificar las unidades de medida. Asimismo, los registros con más de una unidad o sin unidad se asignarán como NA, ya que son ambiguos y podrían afectar el análisis.

```

# Extraemos número
datos["valor"] = datos["bag_weight"].str.extract(r"(\d+)").astype(float)

# Creamos la nueva columna con condiciones
datos["bag_weight2"] = np.select(
    [
        datos["bag_weight"].str.contains("kg", na=False) & ~datos["bag_weight"].str.contains("lbs",
        datos["bag_weight"].str.contains("lbs", na=False) & ~datos["bag_weight"].str.contains("kg",
    ],
    [
        datos["valor"],
        datos["valor"] * 0.45359237
    ],
    default=np.nan
)

datos = datos.drop(columns=["valor"])

# Resultados
print(datos[["bag_weight", "bag_weight2"]].head(10))

```

	bag_weight	bag_weight2
0	60 kg	60.0
1	60 kg	60.0
2	1	NaN
3	60 kg	60.0
4	60 kg	60.0
5	30 kg	30.0
6	69 kg	69.0
7	60 kg	60.0

8	60 kg	60.0
9	60 kg	60.0

Se puede verificar que la nueva columna logra capturar adecuadamente el componente numérico de `bag_weight`.

A partir de esto, surge la pregunta: ¿cuántas observaciones generaron ambigüedad en la construcción de esta variable?

Para responderla, se considerarán como casos problemáticos aquellos registros que carecen de unidad de medida o que presentan más de una unidad.

```
ambigüedad = datos[
    datos["bag_weight"].notna() & (
        datos["bag_weight"].str.match(r"^[0-9]+$") |    # sin unidad
        datos["bag_weight"].str.contains(",")           # múltiples unidades
    )
]

# Contamos el número de datos ambiguos.

print(len(ambigüedad))
```

29

Se observa que, tanto en R como en Python, se identificaron 29 observaciones que presentarían ambigüedad en la conversión de la variable `bag_weight` a formato numérico.

Es importante mencionar que se consideraron ambiguos aquellos registros que no especifican unidad de medida o que presentan más de una unidad en un mismo valor. Los valores atípicos o iguales a cero no se clasificaron como ambiguos, ya que corresponden a problemas de calidad del dato y no de formato.

Actividad 3:

Crear dos columnas llamadas `method1` y `method2` que se basen en los valores de la columna `processing_method`, dividiendo en dos partes los valores dicha columna. ¿Cuántas observaciones llevaron a ambigüedad para crear esta nueva columna?

Se comienza con la implementación en R.

Realizaremos un análisis inicial de la columna `processing_method` con el objetivo de analizar su estructura antes de dividir sus valores.

```
# Datos únicos de la columna processing_method.
unique(datos$processing_method)
```

```
[1] "Washed / Wet"      NA
[3] "Natural / Dry"     "Pulped natural / honey"
[5] "Semi-washed / Semi-pulped" "Other"
```

Podemos notar que algunos registros contienen valores compuestos, como "Semi-washed", así como valores NA. Sin embargo, todos los registros válidos presentan el separador " / ", lo que permite continuar con la siguiente transformación.

```
# Dividimos los valores de la columna processing_method en dos partes.
separacion <- str_split_fixed(
  datos$processing_method,
  " / ",
  n = 2
)

# Creamos las nuevas columnas.
datos <- datos |>
  mutate(
    method1 = separacion[, 1],
    method2 = separacion[, 2]
  )

# Vemos los resultados obtenidos.
datos |>
  select(processing_method, method1, method2) |>
  head(10)
```

```
# A tibble: 10 × 3
  processing_method method1 method2
  <chr>             <chr>   <chr>
1 Washed / Wet     Washed  "Wet"
2 Washed / Wet     Washed  "Wet"
3 <NA>             <NA>    ""
4 Natural / Dry    Natural "Dry"
5 Washed / Wet     Washed  "Wet"
6 Natural / Dry    Natural "Dry"
7 Washed / Wet     Washed  "Wet"
8 <NA>             <NA>    ""
9 <NA>             <NA>    ""
10 Natural / Dry   Natural "Dry"
```

Comparamos las columnas `processing_method` , `method1` y `method2` con la finalidad de verificar que la separación se realizó correctamente.

Finalmente, nos preguntamos, ¿Cuántas observaciones llevaron a ambigüedad para crear esta nueva columna?

```
# Filtramos y contamos aquellos datos que no tienen exactamente un " / ".
sum(
```

```
!is.na(datos$processing_method) &
str_count(datos$processing_method, " / ") != 1
)
```

[1] 26

A continuación, se desarrolla el mismo ejercicio en Python.

Como paso previo, se examina la columna `processing_method` para comprender su estructura antes de proceder a la separación de sus valores.

```
print(datos["processing_method"].unique())
```

```
<StringArray>
[
      'Washed / Wet',          nan,
      'Natural / Dry',        'Pulped natural / honey',
      'Semi-washed / Semi-pulped', 'Other']
Length: 6, dtype: str
```

Se observa que algunos registros incluyen valores compuestos, como "Semi-washed", así como casos con valores NA. No obstante, los registros válidos utilizan de forma consistente el separador " / ", lo que facilita proceder con la siguiente transformación.

```
# Dividir en dos partes

separacion = datos["processing_method"].str.split(" / ", n=1, expand=True)

# Crear nuevas columnas

datos["method1"] = separacion[0]
datos["method2"] = separacion[1]

# Ver resultados

print(datos[["processing_method", "method1", "method2"]].head(10))
```

	processing_method	method1	method2
0	Washed / Wet	Washed	Wet
1	Washed / Wet	Washed	Wet
2	NaN	NaN	NaN
3	Natural / Dry	Natural	Dry
4	Washed / Wet	Washed	Wet
5	Natural / Dry	Natural	Dry
6	Washed / Wet	Washed	Wet
7	NaN	NaN	NaN
8	NaN	NaN	NaN
9	Natural / Dry	Natural	Dry

Se comparan las columnas `processing_method` , `method1` y `method2` con el fin de verificar que la separación se haya realizado correctamente.

Finalmente, se plantea la siguiente pregunta: ¿cuántas observaciones generaron ambigüedad al construir estas nuevas columnas?

```
# Contar observaciones ambiguas

ambigüedad_count = (
    datos["processing_method"].notna() &
    (datos["processing_method"].str.count(" / ") != 1)
).sum()

print(ambigüedad_count)
```

26

Para esta actividad, tanto en R como en Python evaluamos la estructura de la variable `processing_method` mediante el conteo de ocurrencias del separador " / ". Se identificaron 26 observaciones que no presentan exactamente dicho separador, lo que impide dividirlos de forma clara en dos. Esto indica que, aunque la mayoría de los registros siguen un formato consistente, existe un conjunto de datos con estructuras que pueden generar ambigüedad al construir las columnas `method1` y `method2`.

Actividad 4:

Crear tres columnas llamadas `expiration_day`, `expiration_month` y `expiration_year` que se basen en los valores de la columna `expiration`. ¿Cuántas observaciones llevaron a ambigüedad para crear estas nuevas columnas?

Se comienza nuevamente con la implementación en R.

Como primer paso, se analiza la columna `expiration` con el fin de identificar la estructura de sus registros.

```
# Datos únicos de la columna expiration.
head(unique(datos$expiration),20)
```

```
[1] "April 3rd, 2016"      "May 31st, 2011"      "March 25th, 2016"
[4] "September 3rd, 2014"  "September 17th, 2013" "September 2nd, 2011"
[7] "March 29th, 2016"     "March 26th, 2016"     "March 12th, 2016"
[10] "August 31st, 2011"    "June 16th, 2011"     "April 7th, 2017"
[13] "March 24th, 2016"     "April 2nd, 2015"      "July 26th, 2013"
[16] "December 2nd, 2012"   "June 26th, 2015"      "May 18th, 2018"
[19] "May 18th, 2017"       "May 18th, 2011"
```

Podemos observar que los registros siguen el formato "mes día, año", donde el día incluye sufijos como "st", "nd", "rd" o "th". Por lo tanto, la separación puede realizarse de la siguiente manera.

```
# Hacemos una limpieza previa de la columna expiration.
datos <- datos |>
  mutate(
    expiration_limpia = str_replace_all(expiration, "st|nd|rd|th", ""),
    expiration_limpia = str_replace_all(expiration_limpia, ",", "")
  )

fecha_expiration <- str_split_fixed(datos$expiration_limpia, " ", 3)

# Creamos las nuevas columnas.
datos <- datos |>
  mutate(
    expiration_month = fecha_expiration[, 1],
    expiration_day   = as.numeric(fecha_expiration[, 2]),
    expiration_year  = as.numeric(fecha_expiration[, 3])
  )

# Vemos los resultados obtenidos.
datos |>
  select(expiration, expiration_month, expiration_day, expiration_year) |>
  head(10)
```

A tibble: 10 × 4

expiration	expiration_month	expiration_day	expiration_year
<chr>	<chr>	<dbl>	<dbl>
1 April 3rd, 2016	April	3	2016
2 April 3rd, 2016	April	3	2016
3 May 31st, 2011	May	31	2011
4 March 25th, 2016	March	25	2016
5 April 3rd, 2016	April	3	2016
6 September 3rd, 2014	September	3	2014
7 September 17th, 2013	September	17	2013
8 September 2nd, 2011	September	2	2011
9 September 2nd, 2011	September	2	2011
10 March 29th, 2016	March	29	2016

Al igual que en la actividad 3, comparamos las columnas `expiration`, `expiration_month`, `expiration_day` y `expiration_year` con la finalidad de verificar que la separación se realizó correctamente.

Ahora nos preguntamos, ¿Cuántas observaciones llevaron a ambigüedad para crear estas nuevas columnas?

```
# Filtramos y contamos aquellos registros que no son NA y que no cumplen con el formato esperado.
sum(
  !is.na(datos$expiration) &
  !str_detect(datos$expiration, "^[A-Za-z]+ [0-9]{1,2}(st|nd|rd|th), [0-9]{4}$")
)
```

[1] 0

A continuación, se desarrolla el ejercicio en Python.

Como paso previo, se examina la columna `expiration` para comprender la estructura de sus registros.

```
# Datos únicos de la columna expiration (primeros 20)

print(pd.Series(datos["expiration"].unique()).head(20))
```

```
0      April 3rd, 2016
1      May 31st, 2011
2      March 25th, 2016
3      September 3rd, 2014
4      September 17th, 2013
5      September 2nd, 2011
6      March 29th, 2016
7      March 26th, 2016
8      March 12th, 2016
9      August 31st, 2011
10     June 16th, 2011
11     April 7th, 2017
12     March 24th, 2016
13     April 2nd, 2015
14     July 26th, 2013
15     December 2nd, 2012
16     June 26th, 2015
17     May 18th, 2018
18     May 18th, 2017
19     May 18th, 2011
dtype: str
```

Se identifica que los registros presentan el formato “mes día, año”, en el que el día incorpora sufijos como “st”, “nd”, “rd” o “th”. Con base en esta estructura, es posible realizar la separación de la siguiente forma.

```
# Limpieza de la columna expiration

datos["expiration_limpia"] = (
    datos["expiration"]
    .str.replace(r"st|nd|rd|th", "", regex=True)
    .str.replace(",", "", regex=False)
)

# Separar en partes

fecha_expiration = datos["expiration_limpia"].str.split(" ", n=2, expand=True)

# Crear nuevas columnas

datos["expiration_month"] = fecha_expiration[0]
datos["expiration_day"] = pd.to_numeric(fecha_expiration[1], errors="coerce")
datos["expiration_year"] = pd.to_numeric(fecha_expiration[2], errors="coerce")
```

```
# Ver resultados
```

```
print(datos[["expiration", "expiration_month", "expiration_day", "expiration_year"]].head(10))
```

	expiration	expiration_month	expiration_day	expiration_year
0	April 3rd, 2016	April	3	2016
1	April 3rd, 2016	April	3	2016
2	May 31st, 2011	May	31	2011
3	March 25th, 2016	March	25	2016
4	April 3rd, 2016	April	3	2016
5	September 3rd, 2014	September	3	2014
6	September 17th, 2013	September	17	2013
7	September 2nd, 2011	September	2	2011
8	September 2nd, 2011	September	2	2011
9	March 29th, 2016	March	29	2016

De manera similar a la actividad 3, se contrastan las columnas `expiration`, `expiration_month`, `expiration_day` y `expiration_year` para comprobar que la separación se haya realizado correctamente.

A partir de ello, surge la siguiente pregunta: ¿cuántas observaciones generaron ambigüedad al construir estas nuevas columnas?

```
# Contar observaciones ambiguas
```

```
ambigüedad_count = (  
    datos["expiration"].notna() &  
    ~datos["expiration"].str.match(r"^[A-Za-z]+ [0-9]{1,2}(st|nd|rd|th), [0-9]{4}$", na=False)  
)<div data-bbox="80 635 90 650" data-label="Text">

```
0
```


```

Dado que, tanto en R como en Python, no se identificaron observaciones con ambigüedad en la variable `expiration`, podemos decir que todos los registros presentaron un formato consistente que nos permitió separarlos por día, mes y año sin ningún problema.

Actividad 5:

Crear dos columnas llamadas `harvest_mes` y `harvest_año` que se basen en los valores de la columna `harvest_year`, dividiendo en dos partes los valores dicha columna. ¿Cuántas observaciones llevaron a ambigüedad para crear esta nueva columna?

Se inicia el ejercicio en R.

Empezamos analizando los registros que contiene la columna `harvest_year` con el objetivo de identificar su formato y así evitar errores al momento de dividirla en dos partes.

```
# Datos únicos de la columna harvest_year.  
unique(datos$harvest_year)
```

```
[1] "2014"          NA  
[3] "2013"          "2012"  
[5] "March 2010"    "Sept 2009 - April 2010"  
[7] "May-August"    "2009/2010"  
[9] "2015"          "2011"  
[11] "2016"          "2015/2016"  
[13] "2010"          "Fall 2009"  
[15] "2017"          "2009 / 2010"  
[17] "2010-2011"     "2009-2010"  
[19] "2009 - 2010"   "2013/2014"  
[21] "2017 / 2018"   "mmm"  
[23] "TEST"          "December 2009-March 2010"  
[25] "2014/2015"     "2011/2012"  
[27] "January 2011"  "4T/10"  
[29] "2016 / 2017"   "23 July 2010"  
[31] "January Through April" "1T/2011"  
[33] "4t/2010"       "4T/2010"  
[35] "August to December" "Mayo a Julio"  
[37] "47/2010"       "Abril - Julio"  
[39] "4t/2011"       "Abril - Julio /2011"  
[41] "Spring 2011 in Colombia." "3T/2011"  
[43] "2016/2017"     "1t/2011"  
[45] "2018"          "4T72010"  
[47] "08/09 crop"
```

Se identificó una amplia diversidad de formatos en la variable `harvest_year`, incluyendo registros con año simple, combinaciones de mes y año, rangos de años, intervalos con meses, descripciones textuales y formatos no estándar.

Así que para hacer la separación se considerarán con “formato correcto” aquellos registros que permiten identificar de manera explícita un año en formato de cuatro dígitos y, en su caso, un mes asociado. Esto incluye registros con formato “mes año”, únicamente año o rangos de años claramente definidos. Por el contrario, se consideran “ambiguos” aquellos registros que no contienen un año identificable o presentan formatos no estandarizados que impiden una separación directa.

```
# Creamos las nuevas columnas.  
datos <- datos |>  
  mutate(  
    harvest_mes = str_extract(  
      harvest_year,  
      regex("January|February|March|April|May|June|July|August|September|October|November|December"  
    ),  
    harvest_anio = str_extract(harvest_year, "\\d{4}")  
  )  
  
# Vemos los resultados obtenidos.
```

```
datos |>
  select(harvest_year, harvest_mes, harvest_anio) |>
  head(200)
```

```
# A tibble: 200 × 3
  harvest_year harvest_mes harvest_anio
  <chr>         <chr>         <chr>
1 2014          <NA>         2014
2 2014          <NA>         2014
3 <NA>          <NA>         <NA>
4 2014          <NA>         2014
5 2014          <NA>         2014
6 2013          <NA>         2013
7 2012          <NA>         2012
8
9 March 2010    March         2010
10 2014         <NA>         2014
# i 190 more rows
```

Podemos observar que, debido a la falta de una estructura homogénea en los datos, se generan múltiples valores NA en las columnas `harvest_mes` y `harvest_anio`.

Por lo que nos preguntamos, ¿Cuántas observaciones llevaron a ambigüedad para crear esta nueva columna?

```
# Filtramos y contamos aquellos registros que no son NA y que no cumplen con el formato esperado.
sum(
  !is.na(datos$harvest_year) &
  str_count(datos$harvest_year, " ") != 1
)
```

```
[1] 1284
```

A continuación, se desarrolla el ejercicio en Python.

Como paso inicial, se examinan los registros de la columna `harvest_year` para identificar su formato y prevenir posibles errores al dividirla en dos componentes.

```
# Datos únicos de la columna harvest_year

print(datos["harvest_year"].unique())
```

```
<StringArray>
[
    '2014',
    '2013',
    'March 2010',
    'May-August',
    '2015',
    nan,
    '2012',
    'Sept 2009 - April 2010',
    '2009/2010',
    '2011',
```

```

'2016', '2015/2016',
'2010', 'Fall 2009',
'2017', '2009 / 2010',
'2010-2011', '2009-2010',
'2009 - 2010', '2013/2014',
'2017 / 2018', 'mmm',
'TEST', 'December 2009-March 2010',
'2014/2015', '2011/2012',
'January 2011', '4T/10',
'2016 / 2017', '23 July 2010',
'January Through April', '1T/2011',
'4t/2010', '4T/2010',
'August to December', 'Mayo a Julio',
'47/2010', 'Abril - Julio',
'4t/2011', 'Abril - Julio /2011',
'Spring 2011 in Colombia.', '3T/2011',
'2016/2017', '1t/2011',
'2018', '4T72010',
'08/09 crop']

```

Length: 47, dtype: str

Se observó una gran variedad de formatos en la variable `harvest_year`, incluyendo registros con un solo año, combinaciones de mes y año, rangos de años, intervalos con meses, descripciones textuales y otros formatos no estandarizados.

Para realizar la separación, se considerarán como “formato válido” aquellos registros en los que sea posible identificar claramente un año en formato de cuatro dígitos y, en su caso, un mes asociado. Esto abarca casos como “mes-año”, únicamente año o rangos de años bien definidos. En cambio, se clasificarán como “ambiguos” aquellos registros que no contienen un año identificable o presentan estructuras no estandarizadas que dificultan una separación directa.

```

# Crear nuevas columnas

datos["harvest_mes"] = datos["harvest_year"].str.extract(
    r"(January|February|March|April|May|June|July|August|September|October|November|December|Jan|
    flags=re.IGNORECASE
)

datos["harvest_anio"] = datos["harvest_year"].str.extract(r"(\d{4})")

# Ver resultados

print(datos[["harvest_year", "harvest_mes", "harvest_anio"]].head(50))

```

	harvest_year	harvest_mes	harvest_anio
0	2014	NaN	2014
1	2014	NaN	2014
2	NaN	NaN	NaN

3	2014	NaN	2014
4	2014	NaN	2014
5	2013	NaN	2013
6	2012	NaN	2012
7	March 2010	March	2010
8	March 2010	March	2010
9	2014	NaN	2014
10	2014	NaN	2014
11	2014	NaN	2014
12	2014	NaN	2014
13	Sept 2009 - April 2010	Sept	2009
14	March 2010	March	2010
15	2014	NaN	2014
16	May-August	May	NaN
17	2009/2010	NaN	2009
18	2015	NaN	2015
19	2014	NaN	2014
20	2014	NaN	2014
21	2014	NaN	2014
22	2012	NaN	2012
23	2014	NaN	2014
24	2013	NaN	2013
25	2011	NaN	2011
26	2009/2010	NaN	2009
27	2013	NaN	2013
28	2016	NaN	2016
29	2015	NaN	2015
30	NaN	NaN	NaN
31	2012	NaN	2012
32	2011	NaN	2011
33	2015/2016	NaN	2015
34	2014	NaN	2014
35	2013	NaN	2013
36	2012	NaN	2012
37	2010	NaN	2010
38	2009/2010	NaN	2009
39	2014	NaN	2014
40	2012	NaN	2012
41	2010	NaN	2010
42	2010	NaN	2010
43	2015	NaN	2015
44	2014	NaN	2014
45	2011	NaN	2011
46	2014	NaN	2014
47	2013	NaN	2013
48	2013	NaN	2013
49	2012	NaN	2012

Se aprecia que la ausencia de una estructura uniforme en los datos da lugar a la aparición de múltiples valores NA en las columnas `harvest_mes` y `harvest_anio`.

Ante esto, surge la siguiente pregunta: ¿cuántas observaciones generaron ambigüedad al construir estas nuevas columnas?

```
# Contar observaciones ambiguas

ambiguedad_count = (
  datos["harvest_year"].notna() &
  (datos["harvest_year"].str.count(" ") != 1)
).sum()

print(ambiguedad_count)
```

1284

En consecuencia, tanto en R como en Python se identificaron 1284 observaciones ambiguas en la columna `harvest_year`. Esto se debe a la presencia de múltiples formatos en los registros, tales como valores que contienen únicamente el año, rangos de años y descripciones textuales. Estas inconsistencias dificultan la separación de la variable en mes y año, ya que no todos los registros presentan una misma estructura.

Actividad 6:

Elabore una visualización con {ggplot2} que identifique alguna relación entre las columnas `total_cup_points`, `acidity` y `color2` de tal forma que se puedan identificar los colores de la variable `color2`. Es decir, debemos ver los colores, "#00FF66", "#CCEBC5" y "#BFFFFFF".

Se comienza el ejercicio con la implementación en R.

```
# Empezamos filtrando los datos necesarios para nuestra gráfica.
ggplot(
  datos |>
  filter(
    !is.na(acidity),
    !is.na(total_cup_points),
    !is.na(color2),
    !(acidity == 0 & total_cup_points == 0) # Eliminamos observaciones atípicas.
  ),

  # Definimos las variables para los ejes: x = acidity, y = total_cup_points.
  aes(x = acidity, y = total_cup_points)
) +

  # Agregamos los puntos.
  geom_point(
    aes(color = color2),
    size = 3,
    alpha = 0.8
  ) +
```

```

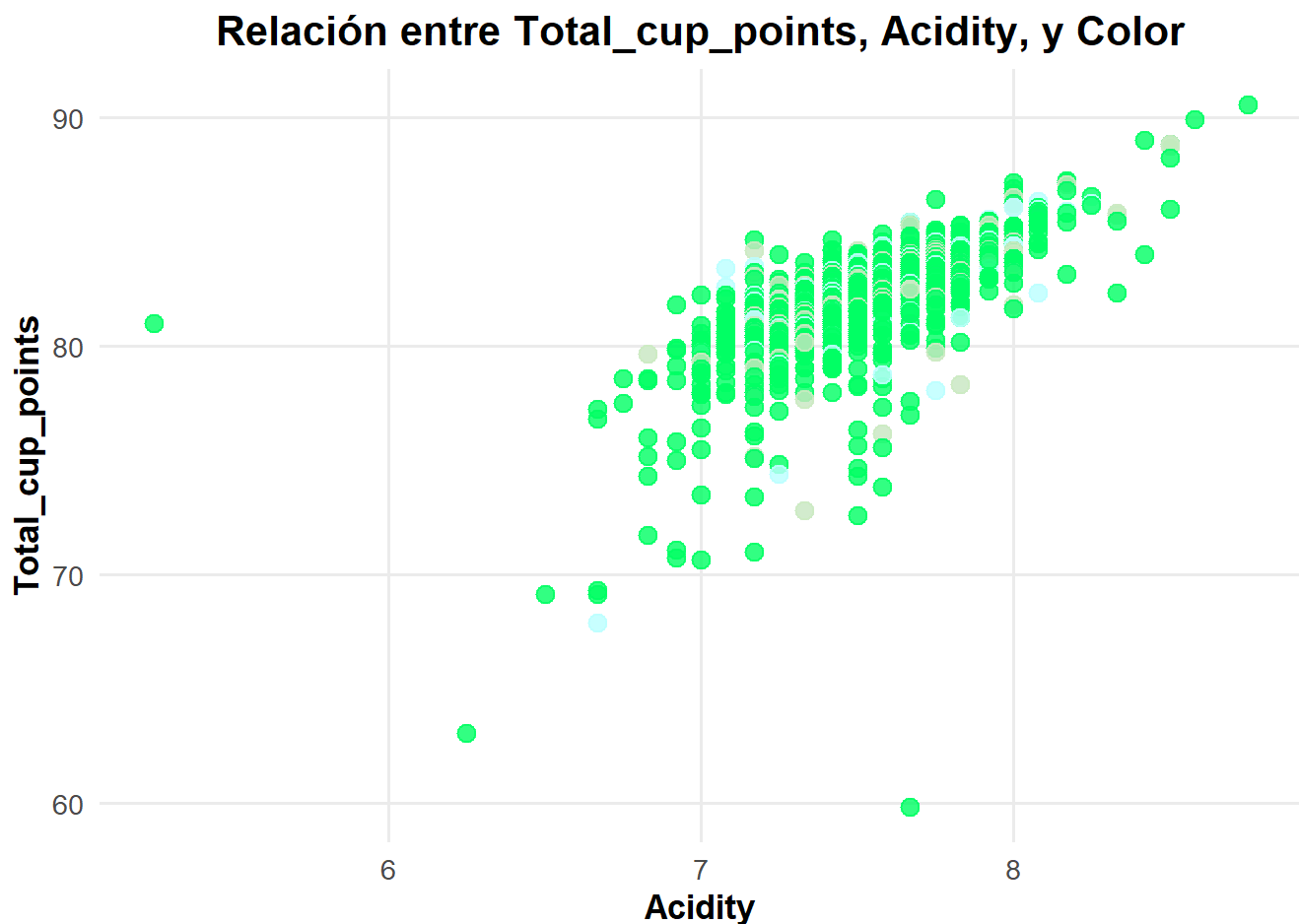
# Usamos directamente los valores de color2 como colores.
scale_color_identity() +

# Agregamos los títulos y etiquetas necesarias a la gráfica.
labs(
  title = "Relación entre Total_cup_points, Acidity, y Color",
  x = "Acidity",
  y = "Total_cup_points"
) +

# Aplicamos un tema minimalista.
theme_minimal(base_size = 13) +

# Modificamos detalles de visualización de la gráfica para que sea entendible.
theme(
  plot.title = element_text(face = "bold", hjust = 0.5),
  plot.subtitle = element_text(hjust = 0.5),
  axis.title = element_text(face = "bold"),
  panel.grid.minor = element_blank()
)

```



A continuación, se reproduce la misma visualización en Python.

```
# Filtrar datos

df_plot = datos[
    datos["acidity"].notna() &
    datos["total_cup_points"].notna() &
    datos["color2"].notna() &
    ~((datos["acidity"] == 0) & (datos["total_cup_points"] == 0))
]

# Gráfica mejorada

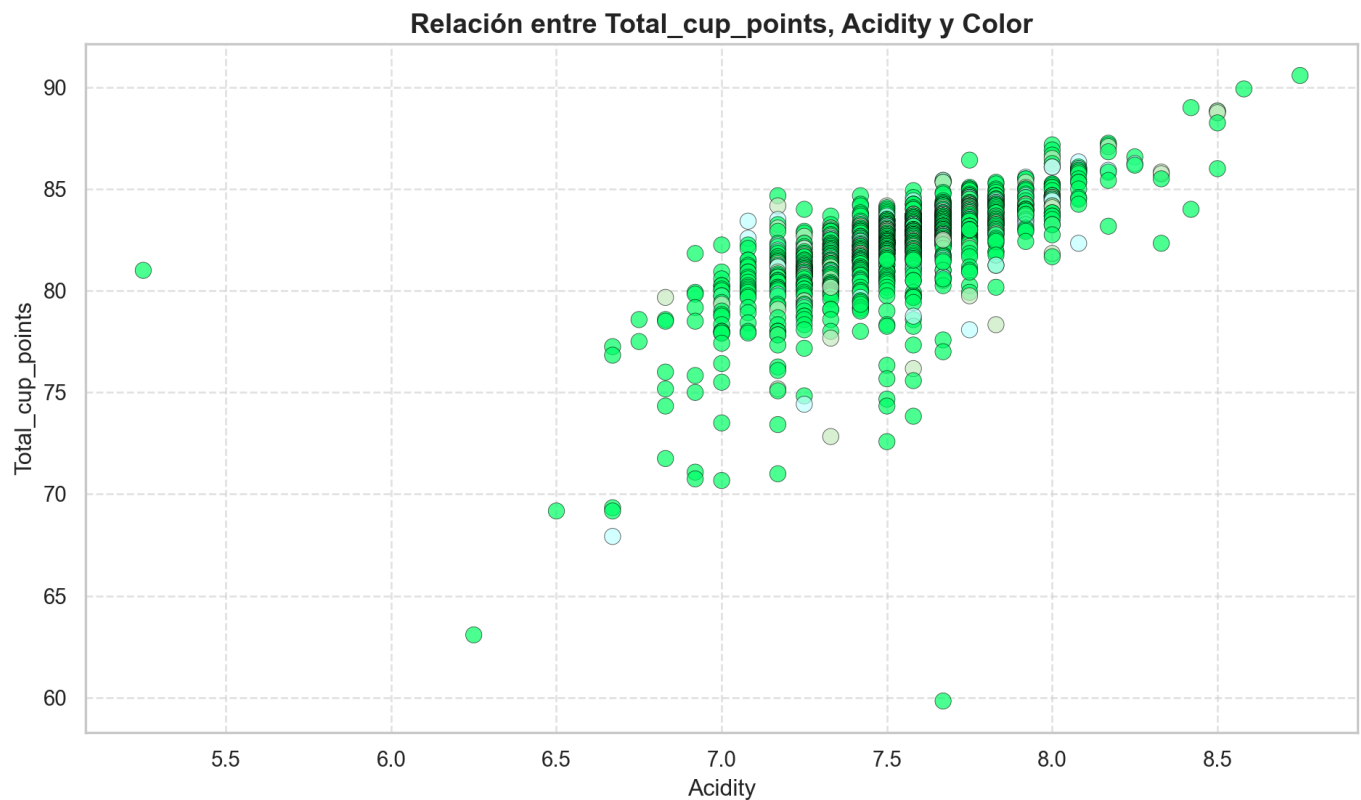
plt.figure(figsize=(10,6))

plt.scatter(
    df_plot["acidity"],
    df_plot["total_cup_points"],
    c=df_plot["color2"],
    s=70,
    alpha=0.7,
    edgecolors="black",
    linewidths=0.3
)

plt.title("Relación entre Total_cup_points, Acidity y Color", fontsize=14, weight="bold")
plt.xlabel("Acidity", fontsize=12)
plt.ylabel("Total_cup_points", fontsize=12)

plt.grid(True, linestyle="--", alpha=0.5)

plt.tight_layout()
plt.show()
```



Es importante notar que, en ambos casos, se decidió eliminar aquellos valores atípicos que podían afectar la visualización de la gráfica, con el fin de hacerla más clara y fácil de entender para el espectador.

Las gráficas muestran una relación positiva entre `acidity` y `total_cup_points`, ya que a mayor acidez, mayor tiende a ser el puntaje del café. Esto es consistente en la mayoría de las observaciones, lo que indica que la acidez es un atributo importante en la evaluación de calidad.

En cuanto a `color2`, se distinguen los colores `"#00FF66"`, `"#CCEBC5"` y `"#BFFFFFF"`, aunque predomina claramente el primero.

Actividad 7:

Elabore una visualización de densidad con {ggplot2} de la variable `bag_weight2` diferenciando a los valores de `species`.

Se inicia el ejercicio con la implementación en R.

```
ggplot(
  datos |>
  filter(
    !is.na(bag_weight2), # Eliminamos los valores faltantes.
    bag_weight2 < 300
  ),
  aes(x = bag_weight2, fill = species)
) +
```

```

geom_density(
  alpha = 0.5,
  color = "black",
  size = 0.3
) +

labs(
  title = "Distribución de Bag_weight2 por Species",
  x = "Bag_weight2",
  y = "Density",
  fill = "species"
) +

coord_cartesian(
  xlim = range(
    datos$bag_weight2[
      !is.na(datos$bag_weight2) & datos$bag_weight2 < 300
    ]
  )
) +

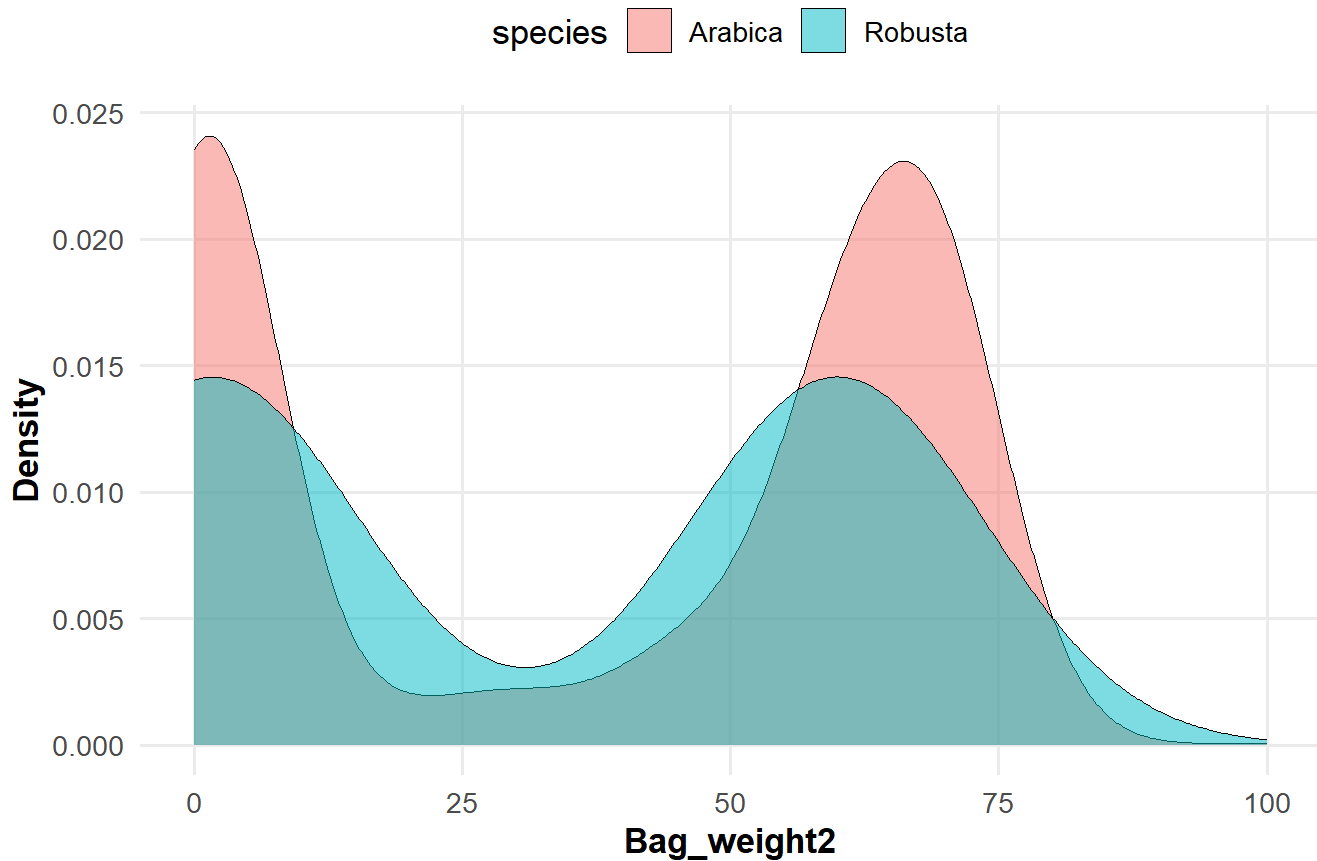
theme_minimal(base_size = 13) +

theme(
  plot.title = element_text(face = "bold", hjust = 0.5),
  axis.title = element_text(face = "bold"),
  legend.position = "top",
  panel.grid.minor = element_blank()
)

```

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
 i Please use `linewidth` instead.

Distribución de Bag_weight2 por Species



A continuación, se presenta la visualización correspondiente en Python.

```
# Filtrar datos
df_plot = datos[
    datos["bag_weight2"].notna() &
    (datos["bag_weight2"] < 300)
]

# Calcular límites dinámicos
xmin = df_plot["bag_weight2"].min()
xmax = df_plot["bag_weight2"].max()

# Gráfica de densidad por especie
plt.figure(figsize=(10,6))

for especie in df_plot["species"].dropna().unique():
    subset = df_plot[df_plot["species"] == especie]
    sns.kdeplot(
        subset["bag_weight2"],
        fill=True,
        alpha=0.4,
        linewidth=1,
        label=especie
    )
```

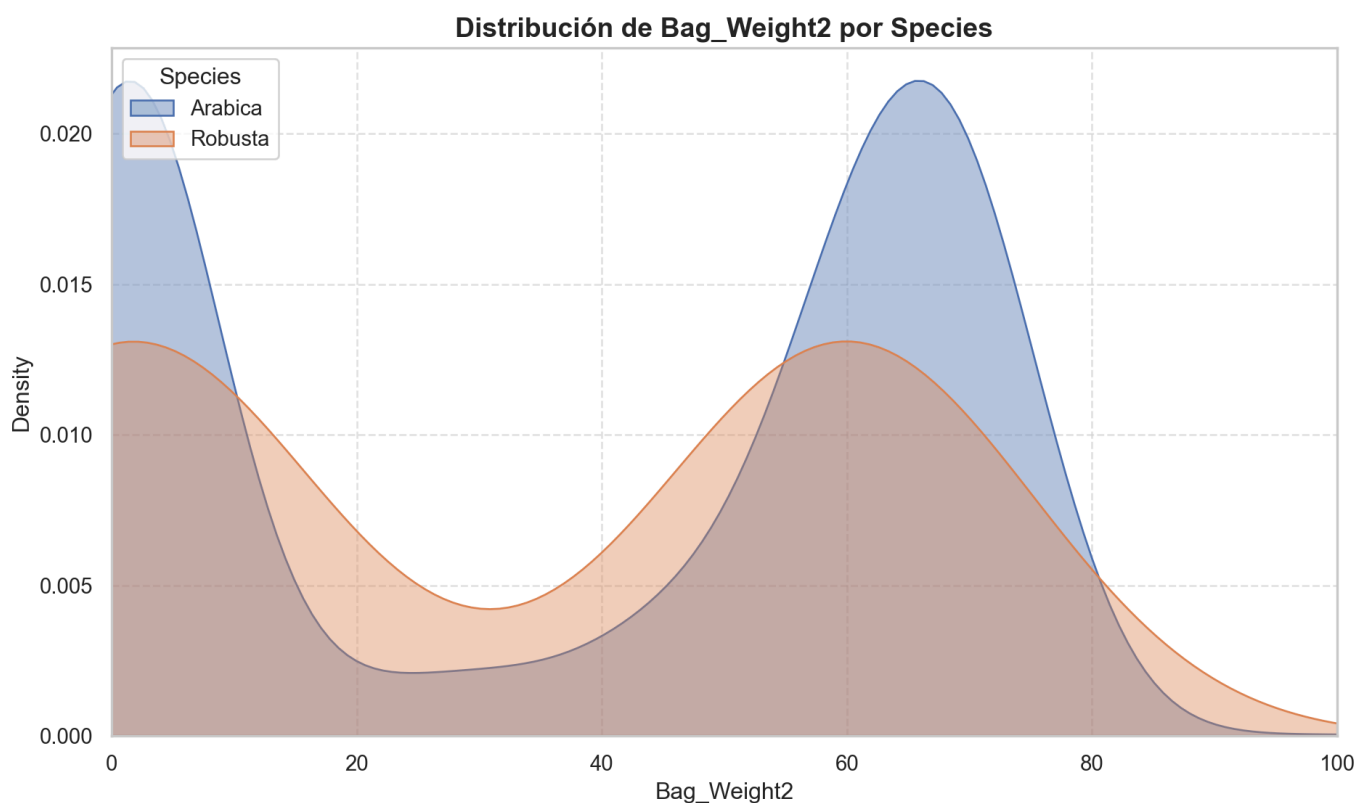
```
# Ajustar eje x dinámicamente
plt.xlim(xmin, xmax)
```

```
(0.0, 100.0)
```

```
plt.title("Distribución de Bag_Weight2 por Species", fontsize=14, weight="bold")
plt.xlabel("Bag_Weight2", fontsize=12)
plt.ylabel("Density", fontsize=12)

plt.legend(title="Species", loc="upper left", bbox_to_anchor=(0, 1))
plt.grid(True, linestyle="--", alpha=0.5)

plt.tight_layout()
plt.show()
```



Es importante destacar que, tanto en R como en Python, se optó por eliminar los valores atípicos (aquellos registros con valores faltantes (NA) y aquellos cuyo valor numérico supera 300), ya que podían distorsionar la visualización. Esta decisión se tomó con el objetivo de obtener una gráfica más clara y fácil de interpretar.

La variable `bag_weight2` presenta una distribución bimodal en ambas especies, con concentraciones en valores bajos y entre 60–80. Aunque Arabica tiende a concentrarse en pesos más altos y Robusta en valores más bajos, existe una amplia superposición entre ambas.

En la "vida real", esto indica que el peso de las bolsas no es un criterio confiable para diferenciar entre Arabica y Robusta, ya que ambas se comercializan en rangos similares y el tamaño del empaque depende

más de decisiones de mercado que del tipo de café.

Actividad 8:

Elabore una visualización que relacione el año/mes de expiración con el `total_cup_points` sólo de los granos mexicanos, brasileños, colombianos y guatemaltecos.

Se comienza el ejercicio con la implementación en R.

```
# Empezamos ordenando los meses para que aparezcan en orden cronológico.
datos_plot <- datos |>
  filter(
    country_of_origin %in% c("Mexico", "Brazil", "Colombia", "Guatemala"),
    !is.na(expiration_year),
    !is.na(expiration_month),
    !is.na(total_cup_points)
  ) |>
  mutate(

    # Convertimos la fecha tipo "YYYY-MM".
    fecha = ymd(paste(expiration_year, expiration_month, "01", sep = "-"))
  ) |>
  group_by(country_of_origin, fecha) |>
  summarise(
    total_cup_points = mean(total_cup_points, na.rm = TRUE),
    .groups = "drop"
  )

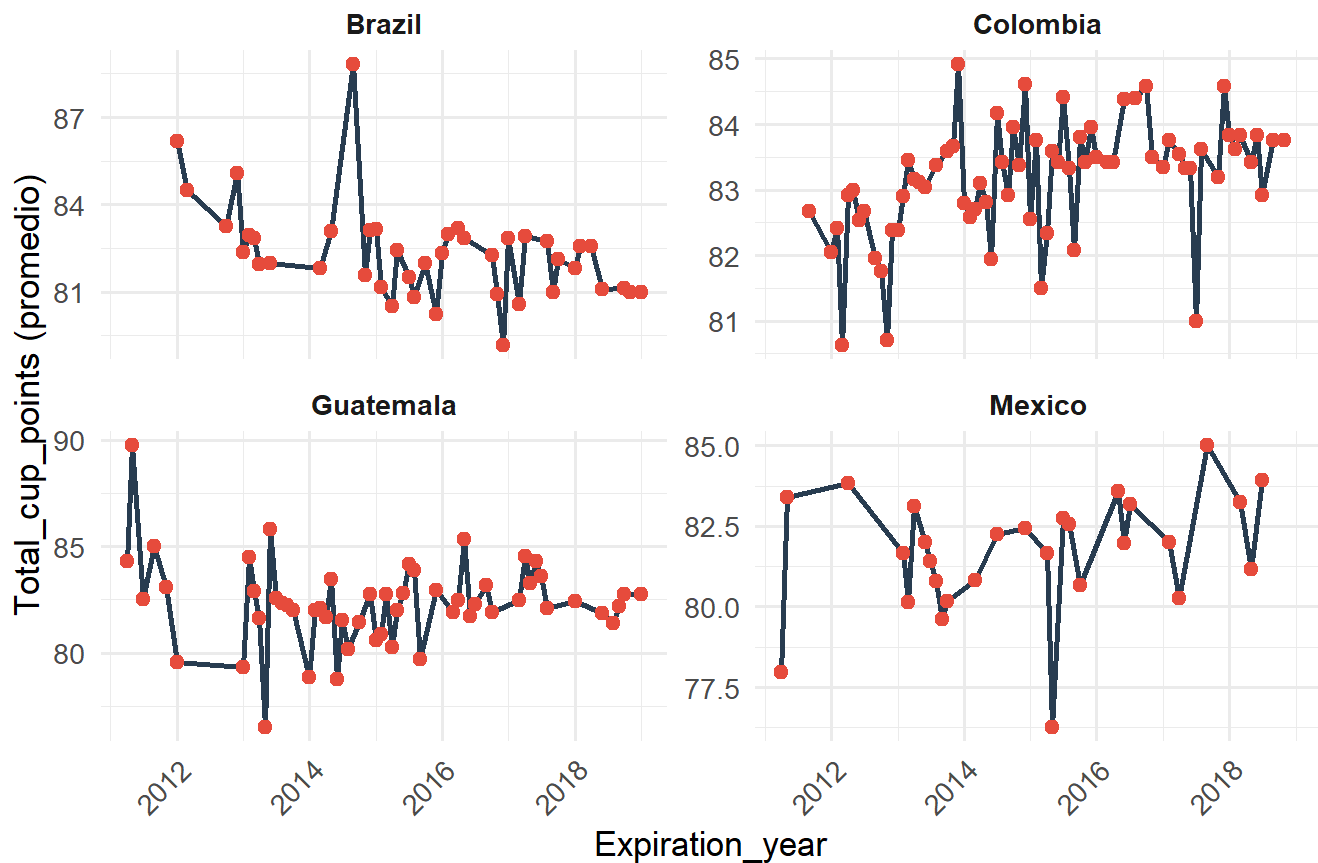
# Generamos la gráfica a partir de los datos previamente modificados.
ggplot(datos_plot, aes(x = fecha, y = total_cup_points)) +
  geom_line(size = 1, color = "#2C3E50") +
  geom_point(size = 2, color = "#E74C3C") +
  facet_wrap(~country_of_origin, scales = "free_y") +
  labs(
    title = "Evolución del puntaje de calidad del café por país",
    x = "Expiration_year",
    y = "Total_cup_points (promedio)"
  ) +

# Aplicamos un tema minimalista.
theme_minimal(base_size = 13) +

# Modificamos detalles de visualización de la gráfica para que sea entendible.
theme(
  strip.text = element_text(face = "bold"),
  plot.title = element_text(face = "bold", size = 16),
  plot.subtitle = element_text(size = 12),
```

```
axis.text.x = element_text(angle = 45, hjust = 1)
)
```

Evolución del puntaje de calidad del café por país



A continuación, se muestra la visualización correspondiente en Python.

```
# Filtrar datos

df_plot = datos[
    datos["country_of_origin"].isin(["Mexico", "Brazil", "Colombia", "Guatemala"]) &
    datos["expiration_year"].notna() &
    datos["expiration_month"].notna() &
    datos["total_cup_points"].notna()
].copy()

df_plot["fecha"] = pd.to_datetime(
    df_plot["expiration_year"].astype(str) + "-" +
    df_plot["expiration_month"] + "-01",
    errors="coerce"
)
```

<string>:3: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.

```

# Agrupar y promediar

df_plot = (
    df_plot
    .groupby(["country_of_origin", "fecha"], as_index=False)["total_cup_points"]
    .mean()
)

countries = df_plot["country_of_origin"].unique()

fig, axes = plt.subplots(len(countries), 1, figsize=(10, 8), sharex=True)

if len(countries) == 1:
    axes = [axes]

for ax, country in zip(axes, countries):
    subset = df_plot[df_plot["country_of_origin"] == country]

    ax.plot(subset["fecha"], subset["total_cup_points"], linewidth=1.5)
    ax.scatter(subset["fecha"], subset["total_cup_points"], s=30)

    ax.set_title(country, fontsize=11, weight="bold")
    ax.grid(True, linestyle="--", alpha=0.5)

fig.suptitle("Evolución del puntaje de calidad del café por país", fontsize=14, weight="bold")
axes[-1].set_xlabel("Expiration_year", fontsize=11)

for ax in axes:
    ax.set_ylabel("Total_cup_points", fontsize=10)

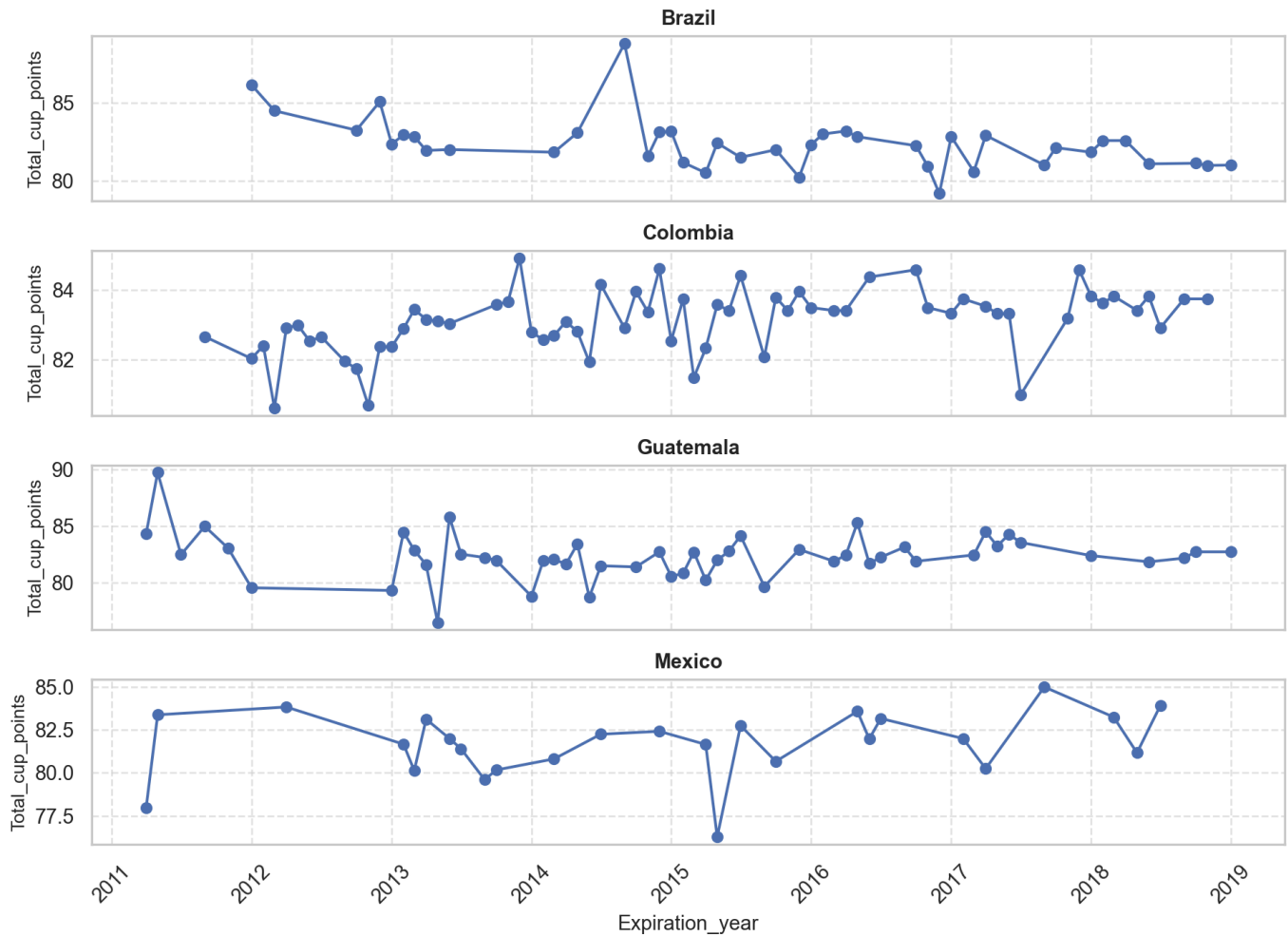
plt.xticks(rotation=45)

(array([14610., 14975., 15340., 15706., 16071., 16436., 16801., 17167.,
        17532., 17897.]), [Text(14610.0, 0, '2010'), Text(14975.0, 0, '2011'), Text(15340.0, 0,
'2012'), Text(15706.0, 0, '2013'), Text(16071.0, 0, '2014'), Text(16436.0, 0, '2015'),
Text(16801.0, 0, '2016'), Text(17167.0, 0, '2017'), Text(17532.0, 0, '2018'), Text(17897.0, 0,
'2019')])

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

```

Evolución del puntaje de calidad del café por país



Tanto en R como en Python, la gráfica indica que no existe una relación clara entre el año/mes de expiración y el puntaje del café, ya que los valores se mantienen relativamente estables en el tiempo.

En cambio, las diferencias más relevantes se observan entre países: Colombia presenta puntajes altos y consistentes, México muestra mayor variabilidad, mientras que Guatemala y Brasil se mantienen en niveles intermedios. En conjunto, esto sugiere que la calidad del café depende más de factores propios de cada país que del tiempo.

Actividad 9:

Elabore una visualización con {ggplot2} que relacione el mes de expiración con el `altitude_mean_meters`, `altitude_low_meters` y `altitude_high_meters` sólo de los granos mexicanos, brasileños, colombianos y guatemaltecos de los años de expiración 2016 y 2017.

Empecemos desarrollando el ejercicio en R.

Antes de comenzar a graficar, haremos un análisis del número de registros disponibles con el fin de evaluar la calidad de los datos y su distribución en las variables de interés, para verificar si existe suficiente

información por mes de expiración para construir una visualización representativa.

```
# Creamos una tabla para hacer una visualización de los datos.
datos_diag_tabla <- datos |>
  mutate(

    # Crear variables de mes incluyendo los NA como categoría.
    expiration_month2 = ifelse(
      is.na(expiration_month),
      "NA",
      as.character(expiration_month)
    )
  ) |>

# Filtramos los países y años de interés para esta actividad.
filter(
  (country_of_origin == "Mexico" |
   country_of_origin == "Brazil" |
   country_of_origin == "Colombia" |
   country_of_origin == "Guatemala") &
  (expiration_year == 2016 | expiration_year == 2017)
) |>

# Agrupamos por país, año y mes.
group_by(country_of_origin, expiration_year, expiration_month2) |>

# Contamos los registros totales y disponibilidad de altitudes
summarise(
  total_registros = n(),
  con_altitude_mean = sum(!is.na(altitude_mean_meters)),
  con_altitude_low = sum(!is.na(altitude_low_meters)),
  con_altitude_high = sum(!is.na(altitude_high_meters)),
  .groups = "drop"
) |>

# Ordenamos para una mejor lectura.
arrange(country_of_origin, expiration_year, expiration_month2)

datos_diag_tabla
```

A tibble: 48 × 7

	country_of_origin	expiration_year	expiration_month2	total_registros
	<chr>	<dbl>	<chr>	<int>
1	Brazil	2016	April	3
2	Brazil	2016	December	3
3	Brazil	2016	February	1
4	Brazil	2016	January	4
5	Brazil	2016	May	1
6	Brazil	2016	November	2
7	Brazil	2016	October	14

8 Brazil	2017 April	2
9 Brazil	2017 Augu	9
10 Brazil	2017 January	2

```
# i 38 more rows
# i 3 more variables: con_altitude_mean <int>, con_altitude_low <int>,
#   con_altitude_high <int>
```

La tabla de conteo muestra que la variable `expiration_month` presenta una alta proporción de valores faltantes, concentrándose la mayoría de los registros en la categoría "NA" y sólo unos pocos en meses específicos, principalmente mayo.

Aunque las variables de altitud cuentan con mayor disponibilidad de datos, la falta de coincidencia con el mes reduce significativamente el número de observaciones completas, lo que impide un análisis concluyente de su relación.

Por ello, el análisis se realiza de forma exploratoria, utilizando únicamente las observaciones con información completa y representando las altitudes mediante promedios mensuales, dada la escasez de datos por mes.

```
# Empezamos ordenando los meses para que aparezcan en orden cronológico.
datos <- datos |>
  mutate(
    expiration_month = factor(
      expiration_month,
      levels = c(
        "January", "February", "March", "April", "May", "June",
        "July", "August", "September", "October", "November", "December"
      )
    )
  )

# Filtramos los datos necesarios para nuestro gráfico.
datos_filtrados <- datos |>
  filter(
    (country_of_origin == "Mexico" |
     country_of_origin == "Brazil" |
     country_of_origin == "Colombia" |
     country_of_origin == "Guatemala") &
    (expiration_year == 2016 | expiration_year == 2017) &
    !is.na(expiration_month) &
    !is.na(altitude_mean_meters) &
    !is.na(altitude_low_meters) &
    !is.na(altitude_high_meters)
  )

# Resumimos por país, año y mes.
datos_resumen <- datos_filtrados |>
  group_by(country_of_origin, expiration_year, expiration_month) |>
  summarise(
    altitude_mean_meters = mean(altitude_mean_meters, na.rm = TRUE),
```

```

    altitude_low_meters = mean(altitude_low_meters, na.rm = TRUE),
    altitude_high_meters = mean(altitude_high_meters, na.rm = TRUE),
    .groups = "drop"
  )

# Generamos la gráfica a partir de los datos previamente modificados.
ggplot(
  datos_resumen,
  aes(
    x = expiration_month,
    y = altitude_mean_meters,
    color = factor(expiration_year),
    group = expiration_year
  )
) +

# Hacemos un rango de altitud a partir del valor de la variable altitude_low_meters y altitude_
geom_linerange(
  aes(
    ymin = altitude_low_meters,
    ymax = altitude_high_meters
  ),
  linewidth = 0.7,
  alpha = 0.7,
  position = position_dodge(width = 0.3)
) +

# Hacemos una línea de tendencia.
geom_line(linewidth = 1) +

# Agregamos los puntos de la altitud media.
geom_point(size = 2.5) +

# Hacemos una gráfica por país.
facet_wrap(~ country_of_origin, ncol = 2, scales = "fixed") +
labs(
  title = "Altitud de cultivo según mes de expiración",
  x = "Expiration_month",
  y = "Altitude",
  color = "Expiration_year"
) +

# Aplicamos un tema minimalista.
theme_minimal(base_size = 12) +

# Modificamos detalles de visualización de la gráfica para que sea entendible.
theme(
  plot.title = element_text(face = "bold", size = 14),
  plot.subtitle = element_text(size = 11),
  axis.text.x = element_text(angle = 45, hjust = 1),
  strip.text = element_text(face = "bold"),

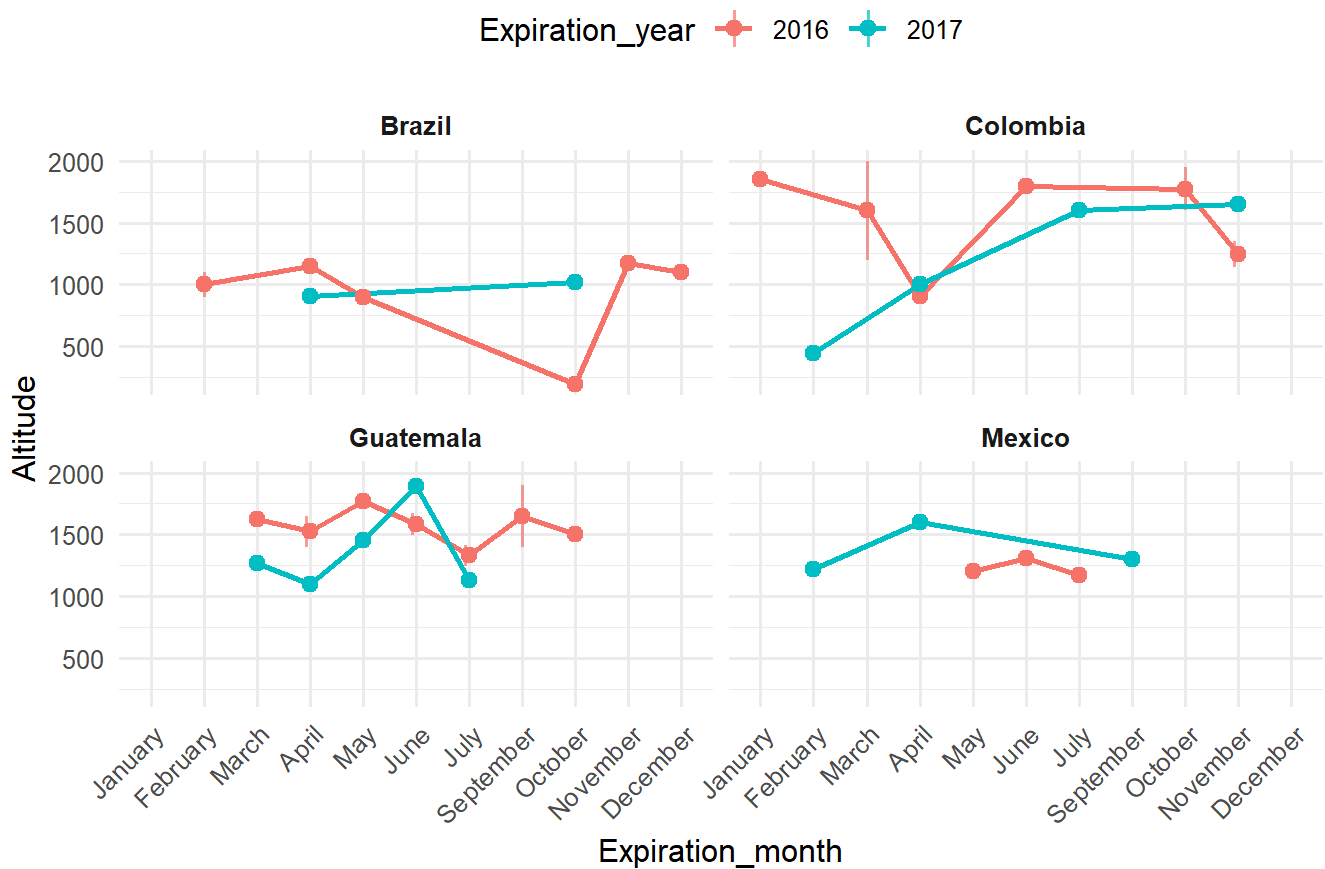
```

```

legend.position = "top"
)

```

Altitud de cultivo según mes de expiración



A continuación, se realiza el ejercicio en Python.

Como paso previo a la visualización, se analiza la cantidad de registros disponibles para evaluar la calidad de los datos y su distribución en las variables de interés. De esta forma, se verifica si hay información suficiente por mes de expiración para construir una gráfica representativa.

```

# Copia de la base
datos_diag_tabla = datos.copy()

# Crear variable incluyendo NA como categoría
datos_diag_tabla["expiration_month2"] = (
    datos_diag_tabla["expiration_month"]
    .fillna("NA")
    .astype(str)
)

# Países de interés
países = ["Mexico", "Brazil", "Colombia", "Guatemala"]

# Filtrar años y países

```

```

datos_diag_tabla = datos_diag_tabla[
    (datos_diag_tabla["country_of_origin"].isin(países)) &
    (datos_diag_tabla["expiration_year"].isin([2016, 2017]))
]

# Tabla diagnóstica
datos_diag_tabla = (
    datos_diag_tabla
    .groupby(
        ["country_of_origin",
         "expiration_year",
         "expiration_month2"],
        dropna=False
    )
    .agg(
        total_registros=("expiration_month2", "size"),
        con_altitude_mean=(
            "altitude_mean_meters",
            lambda x: x.notna().sum()
        ),
        con_altitude_low=(
            "altitude_low_meters",
            lambda x: x.notna().sum()
        ),
        con_altitude_high=(
            "altitude_high_meters",
            lambda x: x.notna().sum()
        )
    )
    .reset_index()
    .sort_values(
        ["country_of_origin",
         "expiration_year",
         "expiration_month2"]
    )
)

datos_diag_tabla

```

	country_of_origin	expiration_year	expiration_month2	total_registros	con_altitude_mean	con_altitude_low	co
0	Brazil	2016	April	3	1	1	1
1	Brazil	2016	December	3	1	1	1
2	Brazil	2016	February	1	1	1	1
3	Brazil	2016	January	4	0	0	0
4	Brazil	2016	May	1	1	1	1
5	Brazil	2016	November	2	2	2	2

	country_of_origin	expiration_year	expiration_month2	total_registros	con_altitude_mean	con_altitude_low	co
6	Brazil	2016	October	14	13	13	13
7	Brazil	2017	April	2	2	2	2
8	Brazil	2017	Augu	9	9	9	9
9	Brazil	2017	January	2	0	0	0
10	Brazil	2017	March	1	0	0	0
11	Brazil	2017	October	2	1	1	1
12	Brazil	2017	September	1	0	0	0
13	Colombia	2016	April	6	5	5	5
14	Colombia	2016	Augu	3	2	2	2
15	Colombia	2016	January	1	1	1	1
16	Colombia	2016	June	2	2	2	2
17	Colombia	2016	March	3	1	1	1
18	Colombia	2016	November	6	5	5	5
19	Colombia	2016	October	2	2	2	2
20	Colombia	2017	April	2	1	1	1
21	Colombia	2017	Augu	3	2	2	2
22	Colombia	2017	December	1	0	0	0
23	Colombia	2017	February	3	2	2	2
24	Colombia	2017	January	3	0	0	0
25	Colombia	2017	July	1	1	1	1
26	Colombia	2017	June	1	0	0	0
27	Colombia	2017	May	1	0	0	0
28	Colombia	2017	November	3	1	1	1
29	Guatemala	2016	April	2	2	2	2
30	Guatemala	2016	July	3	3	3	3
31	Guatemala	2016	June	4	4	4	4
32	Guatemala	2016	March	3	3	3	3
33	Guatemala	2016	May	1	1	1	1
34	Guatemala	2016	October	2	2	2	2
35	Guatemala	2016	September	1	1	1	1
36	Guatemala	2017	April	3	1	1	1
37	Guatemala	2017	Augu	4	4	4	4
38	Guatemala	2017	July	1	1	1	1
39	Guatemala	2017	June	4	4	4	4

	country_of_origin	expiration_year	expiration_month2	total_registros	con_altitude_mean	con_altitude_low	co
40	Guatemala	2017	March	3	3	3	3
41	Guatemala	2017	May	5	4	4	4
42	Mexico	2016	July	3	3	3	3
43	Mexico	2016	June	3	3	3	3
44	Mexico	2016	May	1	1	1	1
45	Mexico	2017	April	1	1	1	1
46	Mexico	2017	February	2	2	2	2
47	Mexico	2017	September	1	1	1	1

La tabla de conteo muestra que `expiration_month` tiene una alta proporción de valores faltantes, con la mayoría de los registros en "NA" y pocos en meses específicos, principalmente mayo.

Aunque las variables de altitud cuentan con más datos, la falta de coincidencia con el mes reduce las observaciones completas, lo que limita el análisis. Por ello, se realiza un enfoque exploratorio utilizando únicamente datos completos y promedios mensuales.

```
plt.close("all")

# Orden cronológico de meses
orden_meses = [
    "January", "February", "March", "April",
    "May", "June", "July", "August",
    "September", "October", "November", "December"
]

datos["expiration_month"] = pd.Categorical(
    datos["expiration_month"],
    categories=orden_meses,
    ordered=True
)
```

<string>:2: Pandas4Warning: Constructing a Categorical with a dtype and values containing non-null entries not in that dtype's categories is deprecated and will raise in a future version.

```
# Filtrar observaciones completas
datos_filtrados = datos[
    (datos["country_of_origin"].isin(paises)) &
    (datos["expiration_year"].isin([2016, 2017])) &
    (datos["expiration_month"].notna()) &
    (datos["altitude_mean_meters"].notna()) &
    (datos["altitude_low_meters"].notna()) &
    (datos["altitude_high_meters"].notna())
]
```

```

# Promedios mensuales
datos_resumen = (
    datos_filtrados
    .groupby(
        ["country_of_origin", "expiration_year", "expiration_month"],
        observed=True
    )
    .agg({
        "altitude_mean_meters": "mean",
        "altitude_low_meters": "mean",
        "altitude_high_meters": "mean"
    })
    .reset_index()
)

plt.style.use("ggplot")
sns.set_context("talk")

g = sns.FacetGrid(
    datos_resumen,
    col="country_of_origin",
    col_wrap=2,
    hue="expiration_year",
    height=5,
    aspect=1.4,
    sharey=True
)

g.map(
    sns.lineplot,
    "expiration_month",
    "altitude_mean_meters",
    marker="o",
    linewidth=2
);

g.add_legend(title="Expiration_year");
g.set_axis_labels("Expiration_month", "Altitude");
g.set_titles("{col_name}");

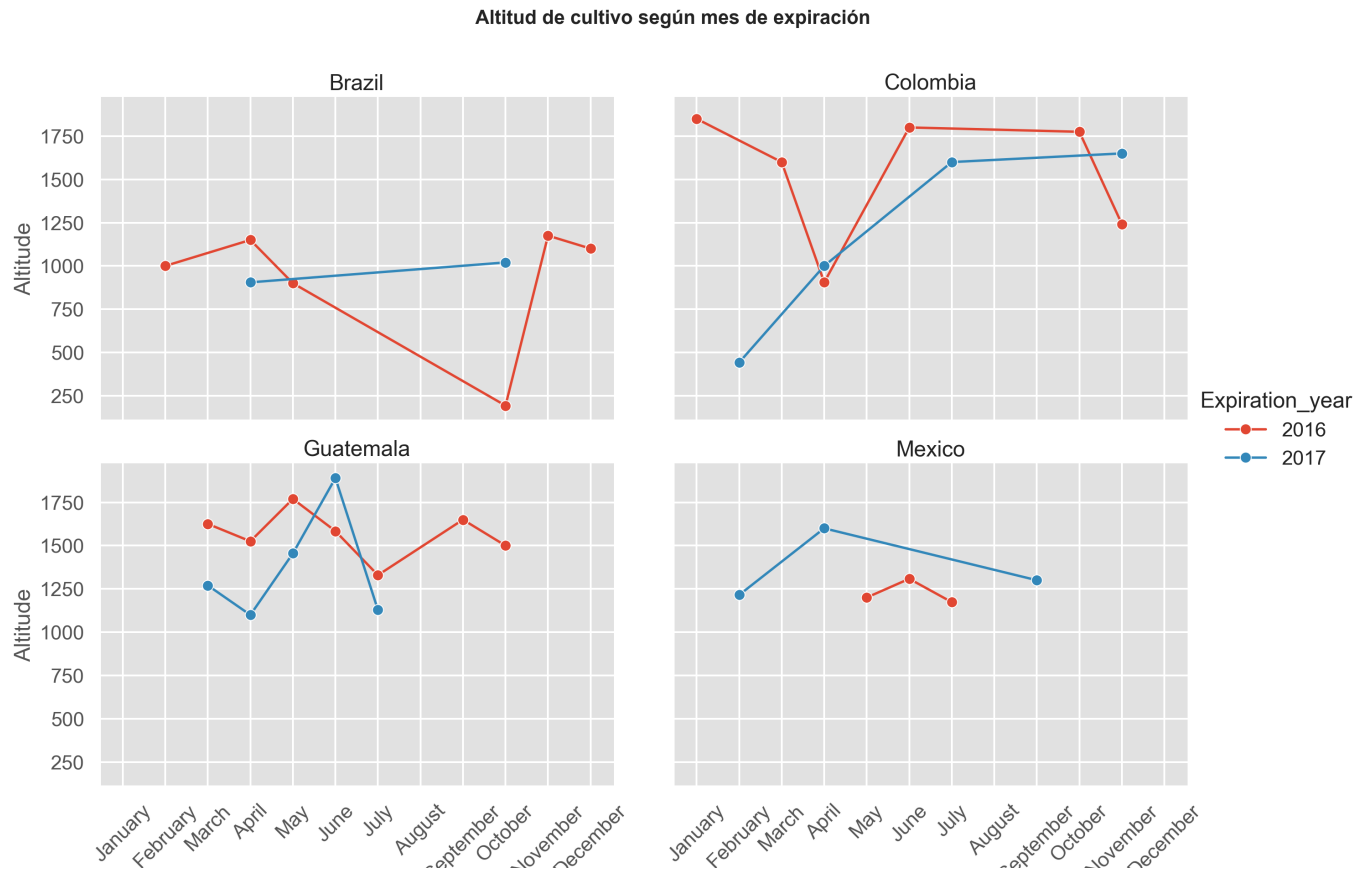
# Rotar meses
for ax in g.axes.flatten():
    ax.tick_params(axis="x", rotation=45)

plt.subplots_adjust(top=0.88)

g.fig.suptitle(
    "Altitud de cultivo según mes de expiración",
    fontsize=16,
    fontweight="bold"
);

```

```
plt.show()
```



```
plt.close(g.fig)
```

Podemos notar que ambas gráficas nos muestra la relación entre el mes de expiración y la altitud de cultivo para México, Brasil, Colombia y Guatemala en 2016 y 2017. En dicho gráfico, las líneas representan la altitud media por año y los segmentos verticales su rango (mínimo-máximo).

En general, no se observa un patrón mensual uniforme entre países, ya que la altitud varía según el origen del grano y el año de expiración. En el caso de Colombia, se aprecia una tendencia más clara en algunos meses; en contraste, México muestra un comportamiento más estable, mientras que Brasil y Guatemala presentan mayor variabilidad. Asimismo, la amplitud del rango de altitud cambia entre meses, lo que indica diferencias en la dispersión de los datos.

Actividad 10:

Elabore una visualización con {ggplot2} que relacione el `aftertaste`, `acidity`, `body` y `species` en un mismo canvas.

Se comienza el ejercicio con la implementación en R.

```

# Empezamos filtrando los datos necesarios para nuestra gráfica.
ggplot(
  datos |>
    filter(
      aftertaste > 0,    # Eliminamos registros con valores no válidos o atípicos.
      acidity > 0
    ),
  aes(
    x = acidity,
    y = aftertaste,
    color = species,
    size = body
  )
) +

# Agrega los puntos (scatter plot).
geom_point(alpha = 0.6) +

# Definimos los colores para cada especie.
scale_color_manual(values = c("darkorange", "steelblue")) +

# Ajustamos el rango de tamaños de los puntos de la gráfica.
scale_size(range = c(1, 5)) +

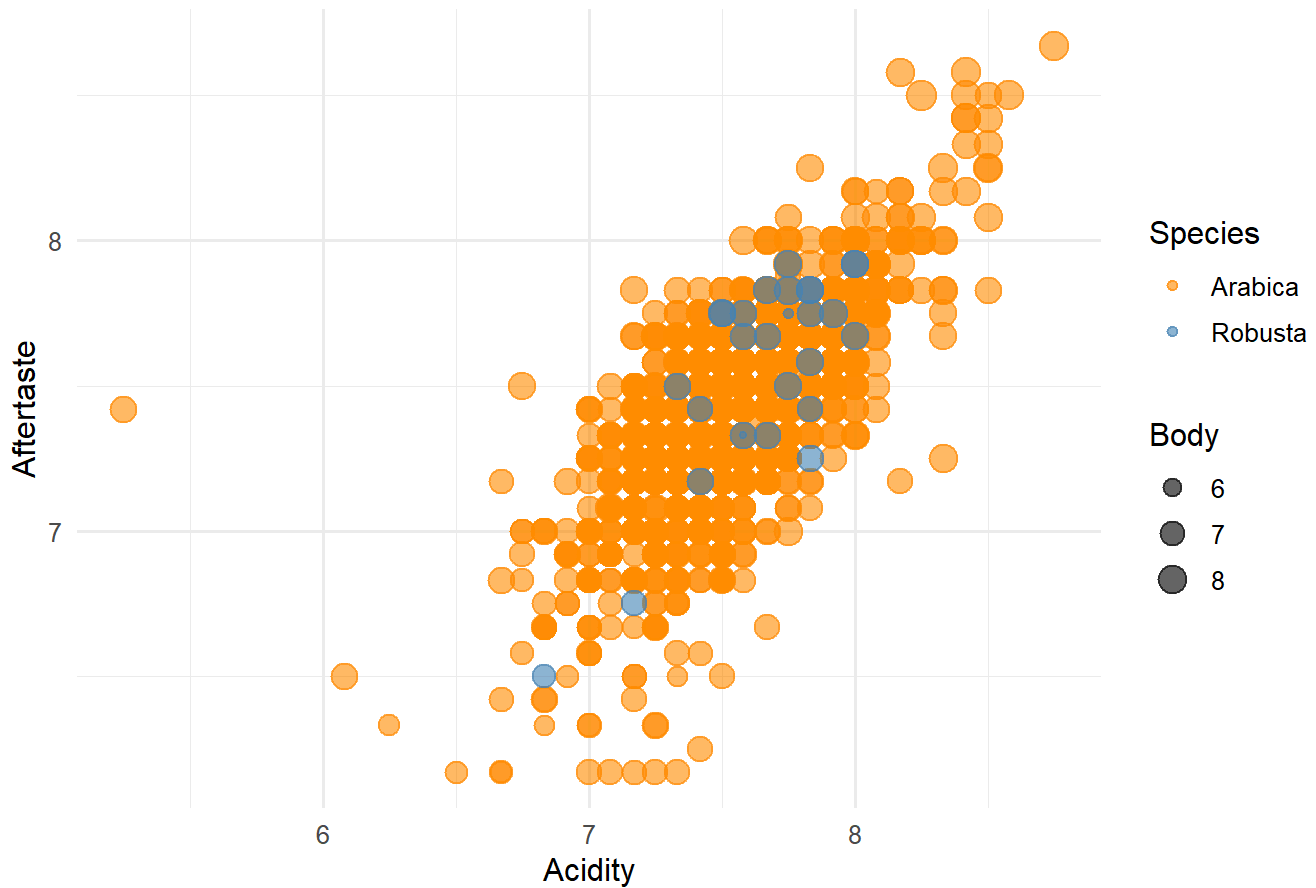
# Agregamos los títulos y etiquetas necesarias a la gráfica
labs(
  title = "Relación entre Aftertaste, Acidity, Body y Species",
  x = "Acidity",
  y = "Aftertaste",
  color = "Species",
  size = "Body"
) +

# Aplica un tema minimalista.
theme_minimal(base_size = 12) +

# Modificamos detalles de visualización de la gráfica para que sea entendible.
theme(
  plot.title = element_text(face = "bold"),
  legend.position = "right"
)

```

Relación entre Aftertaste, Acidity, Body y Species



A continuación, se presenta la gráfica correspondiente en Python.

```
# Filtrar observaciones válidas

datos_filtrados = datos[
    (datos["aftertaste"] > 0) &
    (datos["acidity"] > 0)
].copy()

# Verificar especies

print(datos_filtrados["species"].value_counts())
```

```
species
Arabica    1310
Robusta      28
Name: count, dtype: int64
```

```
plt.style.use("ggplot")
sns.set_context("notebook")

plt.figure(figsize=(10,7))

sns.scatterplot(
```

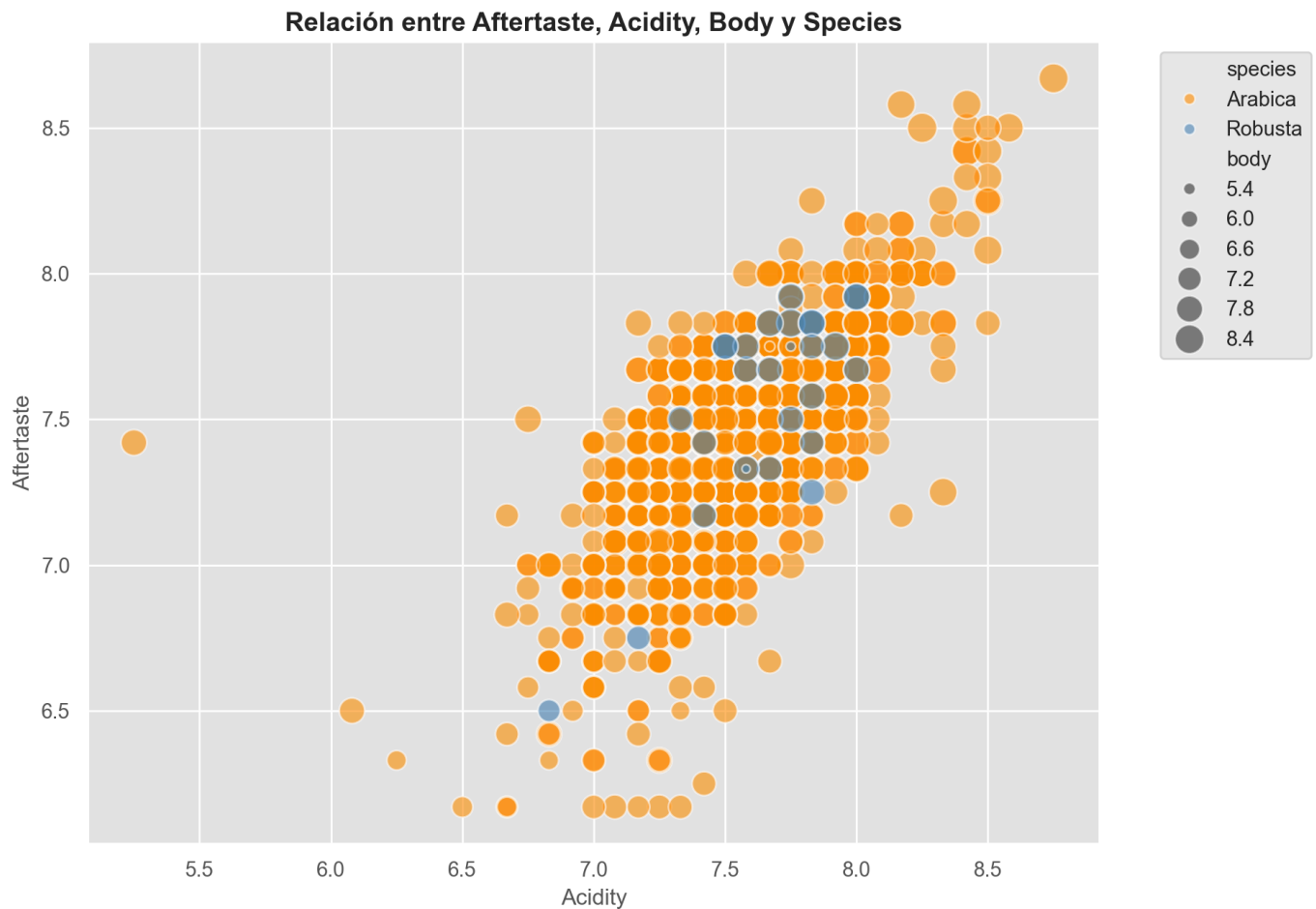
```
data=datos_filtrados,
x="acidity",
y="aftertaste",
hue="species",
size="body",
palette=["darkorange","steelblue"],
sizes=(20,250), # similar a scale_size(range=c(1,5))
alpha=0.6
)

plt.title(
    "Relación entre Aftertaste, Acidity, Body y Species",
    fontsize=14,
    fontweight="bold"
)

plt.xlabel("Acidity")
plt.ylabel("Aftertaste")

plt.legend(
    bbox_to_anchor=(1.05,1),
    loc="upper left"
)

plt.tight_layout()
plt.show()
```



Es importante destacar que, tanto en R como en Python, se decidió eliminar los valores atípicos que podían afectar la visualización de la gráfica, con el fin de hacerla más clara y fácil de interpretar para el espectador.

Podemos observar que las gráficas muestran una relación positiva entre **acidity** y **aftertaste**, es decir, a mayor acidez, mayor es el retrogusto.

Por otro lado, en cuanto a la especie, *Arabica* domina y presenta mayor dispersión, mientras que *Robusta* se concentra en un rango más acotado. Además, los valores más altos de **body** tienden a ubicarse donde **acidity** y **aftertaste** son elevados, lo que sugiere que un mayor cuerpo está asociado a una mayor intensidad.